TD3 Signals

Exercise 1

Program the function sleep (nb_seconds) which puts its caller to sleep for a period of nb_seconds, the calling process resumes its execution at the instruction following the call to sleep.

Exercise 2

It is assumed that a process performs a complex scientific calculation which takes a long time and that the user wishes to know at all times the progress of the work. One solution is to write the progress traces of the calculation to a file, but this solution is not efficient because the file may become very large, and this causes the program to waste time in periodically writing the file. Another more efficient solution is to program Unix signals in a way that instructs the program to display its progress only when desired (upon reception of a signal). Program this solution. For simplicity's sake, assume that the process makes a long loop with a single compute operation, the state of its progress boils down to the iteration counter that you just display on the screen.

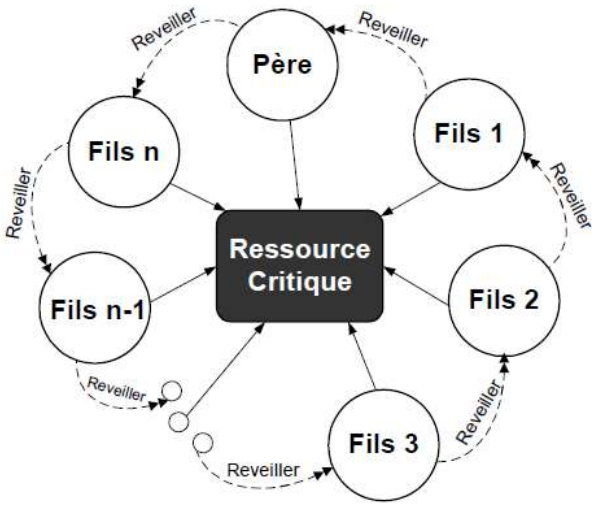
Exercise 3

Management of a critical resource: It is assumed that a process and its children share a critical resource (only one process has the right to access it at a time). To do this, each process, before accessing this resource calls an access authorization procedure (call allouer), and at the end of access calls a release procedure (call liberer) to release the resource. The allocate procedure blocks its caller as long as there is already another process in access, and the free procedure unblocks another process already waiting on allouer. It is assumed that all the processes (father and son) regularly try access to the critical resource, each process obtains its access according to a rotating priority (we imagine that the processes are organized in the form of an oriented ring, where each process awakens its neighbor on the left). The allocate and free procedures are implemented using the pause and kill primitives
respectively.

Aperçu du code permettant l'accès en sécurité à la ressource critique

```
Tanque vrai faire
Debut
      Allouer() ;
      Acceder_ressource() ;
      Liberer() ;
Fin
```

Code exécuter par chaque processus



Exercise 4

Write a program that creates a child that does an endless calculation. The parent process then offers a menu:

1. When the user presses the 's' key, the father process puts his son to sleep.
2. When the user presses the 'r' key, the parent process restarts its child.
    a. When the user presses the 'q' key, the near process kills his child before ending.

Exercise 5

Write a program that creates 5 child processes that make a while 1 loop. The parent process will offer a menu to the user in an endless loop. The menu contains the following:

- Put a son to sleep;
- Wake up a son;
- Finish a son;

b) Edit the threads to display a message when killed. The parent process will display another message if it is killed.

Exercise 6

Write a program that inputs the values of a dynamically allocated n-element integer array. The integer n will be entered on the keyboard. The program displays the value of a tab [i] element where i is entered on the keyboard. In the case of a segmentation error, the program will re-enter the value of i before displaying it.